

Intel® performance hybrid architecture & software optimizations

Part Three: Debugging for Intel performance hybrid architecture

This document is the third in a series of whitepapers that will serve to provide an overview of the Intel performance hybrid architecture.

Authors

Nikhil Rukmabhatla

Software Enabling and
Optimization Engineer

Rajshree Chabukswar

Senior Principal Engineer

Sneha Gohad

Software Enabling and
Optimization Engineer

Michael Chynoweth

Intel Fellow

Introduction

This third whitepaper deals with debugging performance analysis and handling any respective issues that come from performance hybrid architecture and software optimizations. Intel performance hybrid architecture on our 12th Generation Intel Core processors offers many new performance opportunities with its P-cores and E-cores. However, if a developer needs to debug performance issues, this whitepaper provides background and initial support for some of those cases.

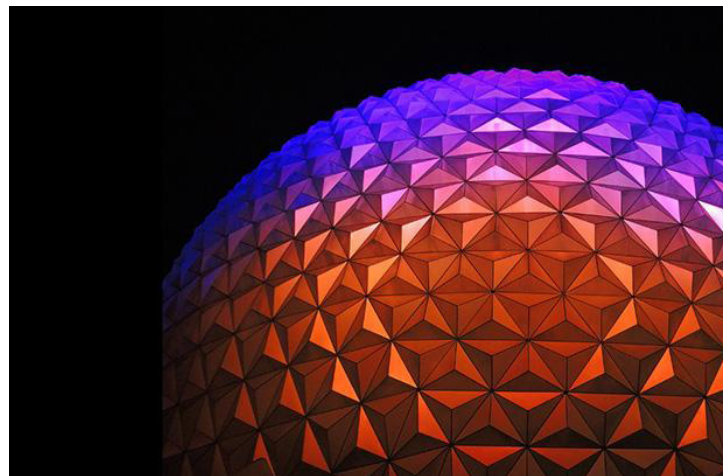


Table of Contents

Introduction	1
Performance Analysis Tools.....	1
Overview of Typeperf.....	2
Case Study #1	3
Case Study # 2	7
Overview of VTune Profiler	8
Case Study # 3	10
Conclusion.....	10
References.....	11

Performance Analysis Tools and Techniques

We have various performance tools we typically use to understand how to improve performance on our performance hybrid architectures and how to isolate and fix problems that can arise. The analysis described in this whitepaper can be used to triage and isolate almost any performance issues using our 12th Generation Intel Core Processors.

While developing the Best Known Methods (BKMs) presented in the previous white papers, we did a lot of analysis on how to get the best performance out of our performance hybrid architecture. Based on that analysis, we developed a checklist of the most important things to look for when debugging. We have included that checklist, case studies, and an overview of tools for debugging in this whitepaper..

Overview of Typeperf

Typeperf is an often under-utilized Windows performance tool that can help monitor both PC and server system-level performance. This tool allows developers and power users to monitor specific or general performance usage and activity by writing that performance data to the command window or log file. Typeperf also does not need any additional installation, since this tool is available on all Windows installations by default. Finally, Typeperf is a good tool to capture overall system-level stats when analyzing the performance of a software stack for the first time.

To use this tool:

1. Open the command prompt
2. Execute the command: `typeperf -cf <input strings> -o %COMPUTERNAME%_perfmon.csv`
 - a. The above command also creates a csv spreadsheet with system stats over time (default is one second)

We recommend that you only use typeperf for capturing critical sections of your workload. Some typical statistics that we capture utilizing TypePerf:

Typeperf Statistic	Rule of Thumb	Notes
\Processor(_Total)\% Processor Time	Tells you the overall CPU utilization.	Anything with >1/logical cores means you could be CPU bound
\Processor(_Total)\% User Time	Usually, a higher user time is best and within applications control	
\Processor(_Total)\% Privileged Time	>20% is always worth reviewing how you are getting into the system. Usually involves #1 system calls, #2 page faults, #3 interrupts, #4 DPCs	Usually want to black box on the kernel with ETL sampling mode
\Processor(_Total)\% Interrupt Time	>1% is a high interrupt time since the top half of the interrupt handler is tagged here	Determine the source of the interrupt from ETL
\Processor(_Total)\% DPC Time	>2% on deferred procedure calls is high and requires investigation	Determine the source of the DPC from ETL
\System\System Calls/sec	>100k per active thread	Have to black box on the kernel to determine the sources of the system call using ETL in sampling mode.
\System\Context Switches/sec	>10k per active thread becomes interesting	The precise mode can tell you the call stack to the context switch
Processor(_Total)\Interrupts/sec	>10k is always interesting to understand the source.	Understand the source of interrupts
\Memory\Page Faults/sec	>50k per active thread but need to determine hard/disk vs. soft/memory	Usually, the majority is Demand Zero Page Faults which are caused by allocating memory from ring0
Memory\Cache Faults/sec	Fault on the file system cache	Find the file being accessed in ETL
\Memory\Pages/sec	Hard page faults look at %disk time	Find the file I/O.
\Memory\Demand Zero Faults/sec	>50k per active thread is interesting. Soft page faults from ring0 allocations.	Find the call stacks to the allocation.
LogicalDisk(_Total)\% Disk Time	>5% becomes interesting to review where I/O is coming from	High disk time
Physical Disk(Total)\Avg. Disk Queue Length	>1 means you are exponentially increasing in I/O handling	Means too many I/Os queued to disk
Processor(_Total)\C1 Transitions/sec	>1k per active thread is interesting	Usually, a thread is explicitly giving up its quanta
Processor(_Total)\C2 Transitions/sec	>1k per active thread is interesting	Usually, a thread is explicitly giving up its quanta
Processor(_Total)\C3 Transitions/sec	>1k per active thread is interesting	Usually, a thread is explicitly giving up its quanta
\Synchronization(_Total)\IPI Send Routine Requests/sec	>10k is problematic. The IPIs are used for scheduling and one core getting is another's attention	
\Synchronization(_Total)\IPI Send Broadcast Requests/sec	>10k is problematic. Broadcast IPIs are usually due to TLB shootdowns but not always	Usually from changing page attributes
\Processor Information(_Total)\% of Maximum Frequency	Reveals the frequency	
\Hyper-V Hypervisor*	VBS/HyperVisor Stat	A large number of stats for virtualization
\Hyper-V Hypervisor Logical Processor(*)*	>5% VBS/HyperVisor Stat	% Time in VMM can become critical if >5%
\Hyper-V Hypervisor Partition(*)*	VBS/HyperVisor Stat	
\Hyper-V Hypervisor Root Partition(*)*	VBS/HyperVisor Stat	
\Hyper-V Hypervisor Root Virtual Processor(*)*	VBS/HyperVisor Stat	
\Hyper-V Hypervisor Virtual Processor(*)*	VBS/HyperVisor Stat	

Table 1. typeperf table

For more information about typeperf, documentation is available at: <https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/typeperf>

Case Study 1: Mitigating High Page Fault rate with typeperf

Page fault rate for a workload can be measured with Typeperf and broken down into hard and soft page faults. When using typeperf, make sure the typeperf input string contains "\Memory\Demand Zero Faults/sec" as well as "\Memory\Page Faults/sec".

We captured several cases where workload performance is impacted due to a high soft page fault rate (greater than 100,000/sec). Analysis of such workloads revealed that these workloads mostly have large frame-sized allocations or are directly calling VirtualAlloc and have obtained zeroed heaps from the operating system. To avoid high page fault performance penalty in such cases, the recommendation is to reuse the pre-allocated memory instead of performing on-demand allocations frequently during an application run.

One example of high page fault workload is the SPECwpc3.1-lammps-chute subtest. The first few seconds of this workload showed unreasonably high page faults (~6 times the average page faults of total execution). When we tried a workload with a proof-of-concept to reuse pre-allocated memory with VirtualAlloc (that pre-allocated memory based on application needs), page faults on this SPECwpc3.1-lammps-chute subtest were reduced significantly. This also improved workload performance.

Overview of Windows Performance Toolkit

The Windows Performance Toolkit is a toolkit that includes tools like the Windows Performance Recorder (WPR) and the Windows Performance Analyzer (WPA). The recorder works to record a session or an event through a log, and the analyzer displays that information for analysis.

To install this toolkit, go to docs.microsoft.com and search with the keywords "Windows Performance Toolkit".

What also comes in this toolkit is Xperf, a command-line tool that can also capture Event Tracing.

Event Tracing is useful because it can demonstrate how workloads are being handled by the targeted system. In this case, the processors. We work to get Event Trace Logs (ETLs) to display through either Xperf or WPR.

Here is an example to showcase how to collect an ETL using WPR. Similarly, the Xperf command-line utility can be used to collect a log. For more information, visit <https://docs.microsoft.com/en-us/windows-hardware/test/wpt/xperf-command-line-reference>

Launch WPR and select profiles depending on which resource to analyze.

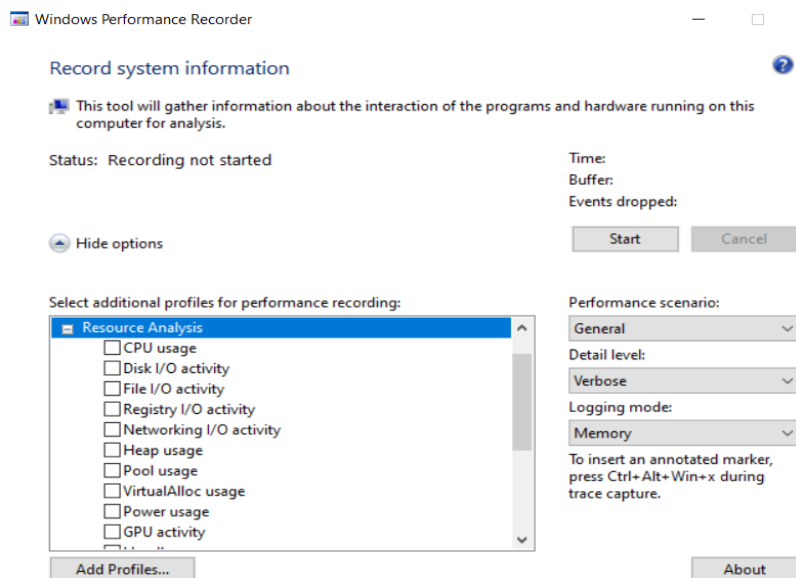


Figure 1. WPA Record system information window

Typically, we select CPU usage and Power Usage for CPU analysis. Optionally also collect I/O activity if the workload is memory bound.

These traces captured with Xperf or WPR can be analyzed using WPA.

The WPA tool creates graphs and data tables (Logs) for Event Tracing. These ETLs can be opened by WPA for analysis, which is useful for doing deep system analysis and identifying performance and power issues.

ETL can also be useful for debugging the following issues:

1. Checking setup
2. CPU utilization and scheduling order
3. Checking parking stats
4. Debugging increased context switch issues and finding switch reasons
5. Background activity and debugging its impact on applications in the foreground
6. Thread Priority and its impact on scheduling
7. Affinity issues
8. Debugging high page fault issue
9. Debugging high system call issue
10. Debugging disk IO issues

Below are some of the BKM's for debugging issues using ETL.

1. Check if Quality of Service (QoS) tagging is working as expected using WPA:

Each thread has QoS assigned either by developers or the OS. The thread QoS level decides performance and power efficiency needs for that thread and based on that OS selects the right core for each thread. Please refer to documentation from MS for more details on QoS levels and classification: <https://docs.microsoft.com/en-us/windows/win32/procthread/quality-of-service>

Using Xperf and WPA tools, developers can trace applications and confirm that QoS tagging looks as expected. Below are steps to setup WPA analysis to understand process/thread level QoS tagging for an application:

- Capture ETL trace using Xperf or Windows Performance monitor profiling for CPU activity.
- Open the ETL trace with the Windows Performance Analyzer and drag the CPU Usage (Precise) graph in the Analysis tab to view the data table. Using the settings wheel icon arrange 'New Thread BamQoSLevel' column next to 'New Process' or 'New Thread ID' column. Below is an example snapshot showing processes running with appropriate QoS levels for maximum performance and efficiency for Microsoft Edge execution.

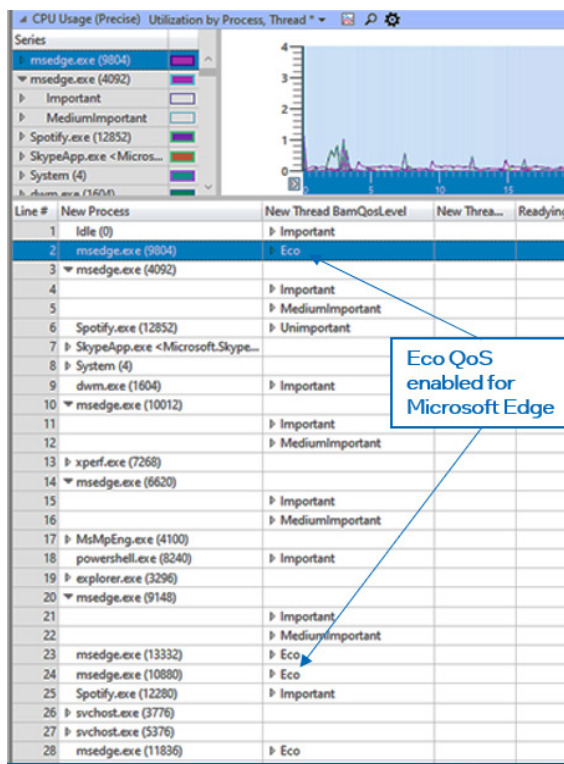


Figure 2. ETL Analysis Tab Showing Eco QoS Enabled

2. How to check Process-/Thread-level priority using WPA

Process/thread priority can be reviewed with the ETL trace analysis. Below are the details with examples regarding ways to carry out this analysis and understand performance impact.

- Capture ETL trace using Xperf or Windows Performance monitor profiling for CPU activity.
- Open the ETL trace with the Windows Performance Analyzer and drag the CPU Usage (Precise) graph in the Analysis tab to view the data table. Using the settings wheel icon arrange columns 'New Thread BamQoSLevel' and 'New In Pri' in the data table as shown in the snapshot below. These two columns will help to understand thread priority and associated QoS level.
- The snapshot below is from a sample trace of SPECworkstation3 HandBrake workload. As we can see in this example, the activity tagged as unimportant is typically LowQoS and Activity tagged as Important is high/medium QoS.

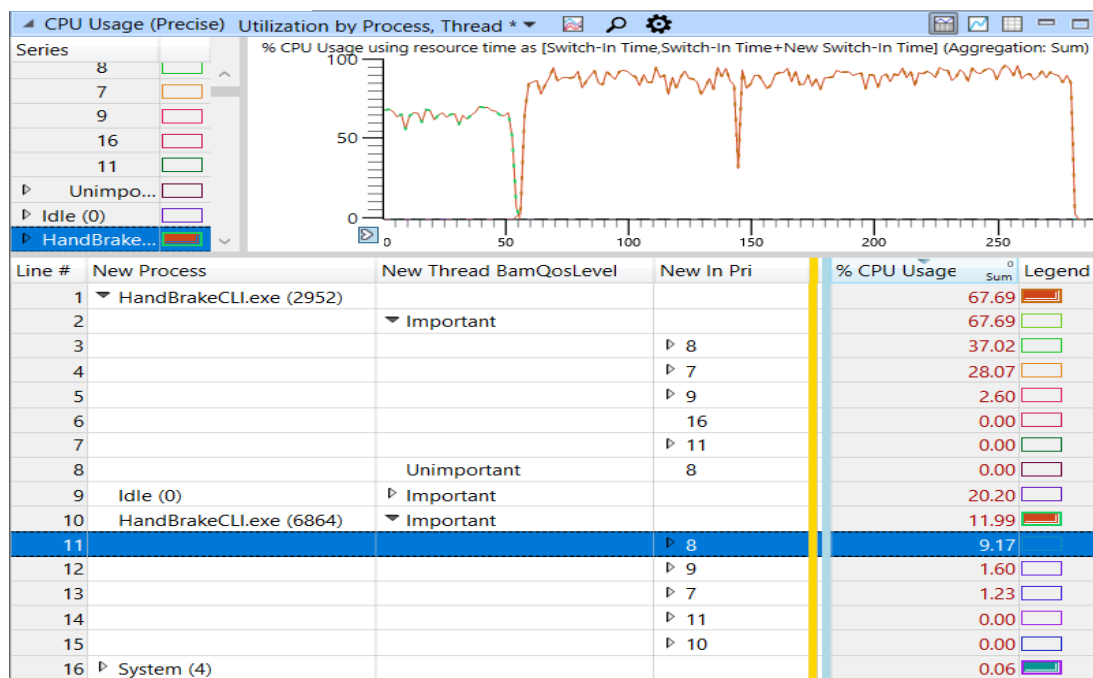


Figure 3. ETL Sample Trace showing QoS Levels and priorities.

- Thread priorities can also be reviewed with CPU Usage (Sampled) analysis. For this, drag CPU Usage (Sampled) graph into the Analysis tab and arrange columns in the data table as shown in the images below. Use this sequence: Process -> Priority -> Thread ID -> CPU. With this analysis setup, we can also understand how threads are scheduled to cores based on the priority levels.

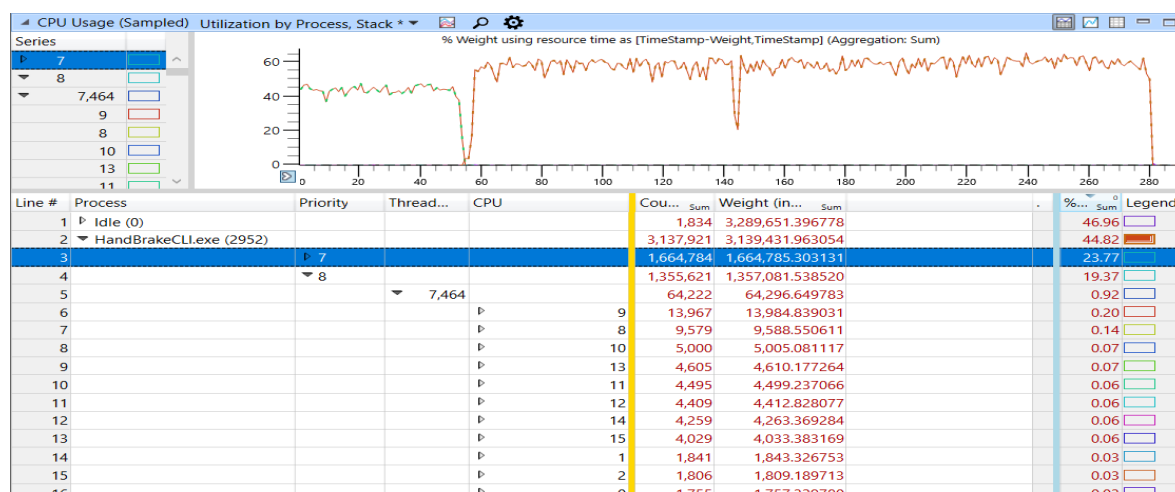


Figure 4. ETL Sample Trace showing thread priority and core scheduling

3. If low QoS is noticed then check if 'Window in Focus' during application runtime using WPA

First, let's understand how workload performance is affected on Windows OS based on whether the application window is in focus or not. Typically, tasks running in the foreground with the application window in focus are considered as tasks contributing to user experience. By default, Windows OS assigns High Quality of Service (QoS) to those foreground tasks unless the developer overrides the OS with a Low Quality of Service assignment through power throttling APIs. Foreground applications also get a priority boost for enhanced performance. When an application runs with multiple windows, it can be useful to verify that the proper application window has focus at given times to help achieve the best possible performance and user experience.

- Using Xperf, capture an ETL trace for workload with the following provider for 'Window in Focus': **Microsoft-Windows-Win32k: 0x402000: 0xff: Stack**
- Open the ETL trace with the Windows Performance Analyzer. Expand System Activity node in Graph Explorer window and drag 'Window in Focus' to analysis tab to view the full-size version of the graph with the associated data table as shown below. You can observe the duration for which a process has a window in focus.

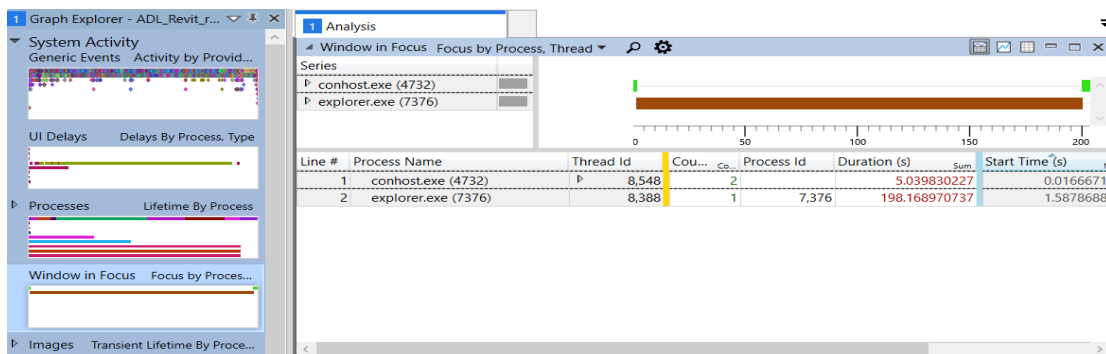


Figure 5. ETL Window in Focus

4. Find increased spins with higher core count to see non-scalable work using WPA

In general, spinning (also known as busy waiting) is a pattern that happens when a process or a thread repeatedly checks for a condition and can potentially waste valuable processor time. Traditionally used delay mechanisms or active spins through continually calling SleepEx, Sleep (0), or SwitchToThread() while a thread waits for work while spinning attempting to give up its quanta can be very expensive. If the workload is using active spins in a critical path then impact on workload scalability can be observed with increases in core counts.

These active spins can be captured using call stack analysis in WPA. Below is an example snapshot from ETL trace analysis with WPA. With the CPU Usage (Sampled) view, we can see the 'NtDelayExecution' function call coming from the ntdll.dll module for the workload. As highlighted in the call stack below, the 'NtDelayExecution' function is called from workload binary executing spins through SleepEx or Sleep APIs.

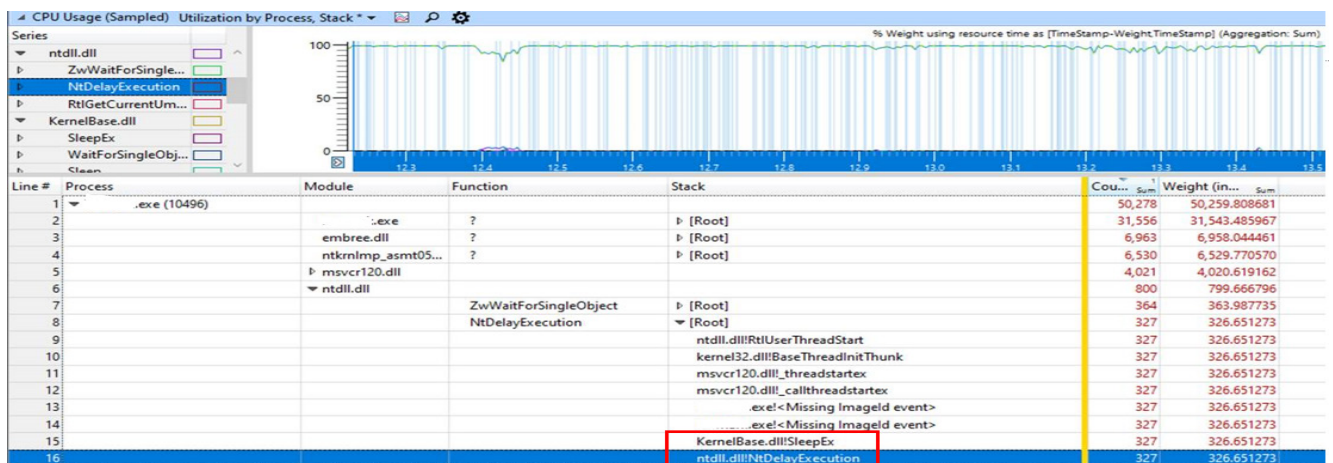


Figure 6. ETL spins call stack

Case Study 2: Hard Affinity Removal Optimization Using ETL Analysis

Hard affinity that is generated by SetProcessAffinityMask() or SetThreadAffinityMask() API can be harmful and can create performance implications on the 12th Generation Intel Core processors by not allowing the OS to decide where to schedule the thread. With ETL analysis of SPECWorkstation3.0-RodiniLifeSci workload, we could optimize one such hard affinity case.

CPU Usage Sampled analysis snapshot below helps understand the workload behavior. We can see that on our 12th Generation Intel Core processors with 8 logical hyper-threaded P-cores and 8 E-cores that workload launches two processes (Highlighted in red box) performing similar work on separate datasets.

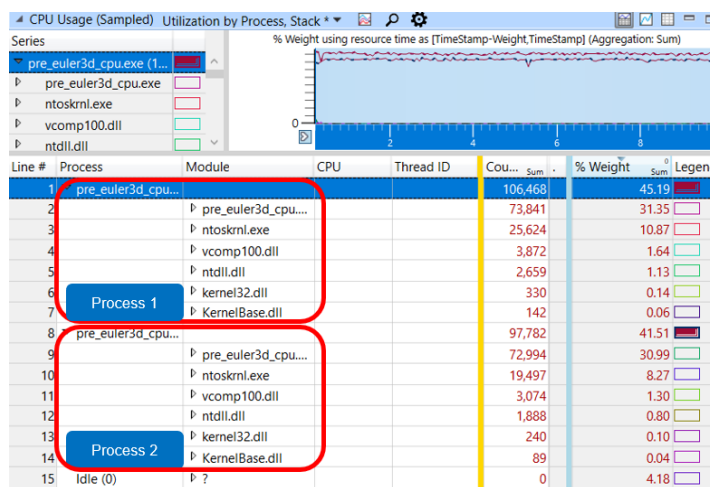


Figure 7. ETL CPU Usage analysis

With both processes being multi-threaded and highly scalable, there is a performance boost with an increased number of cores. As we can see in the next snapshot from CPU Usage (precise) analysis, Process 1 is running tagged to a set of 12 CPUs, which is a combination of 4 P-cores and 8 E-cores. At the same time, Process 2 is running tagged to the remaining 12 CPUs which are all P-cores. This tells us that we have a 'process to core' hard affinity set.

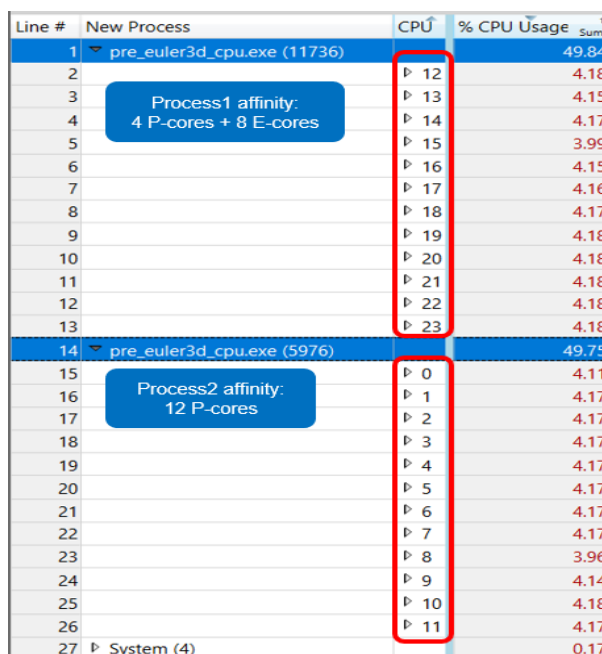


Figure 8. Continued ETL CPU Usage analysis

Below are two snapshots from CPU Usage (precise) analysis side-by-side, for each workload process, after 'process to CPU core' affinity is removed. As we can see in the snapshots, both workload processes are now running on all available CPU cores. With this, we see a good performance for the workload.

Line #	New Process	CPU	% CPU Usage	Sum	Line #	New Process	CPU	% CPU Usage	Sum
1	pre_euler3d_cpu.exe (7184)			49.83	1	pre_euler3d_cpu.exe (7184)			49.83
2		P 6		2.76	2	pre_euler3d_cpu.exe (1836)			49.77
3		P 7		2.75	3		P 19		2.72
4		P 5		2.72	4		P 23		2.72
5		P 4		2.72	5		P 20		2.71
6		P 0		2.70	6		P 22		2.70
7		P 1		2.69	7		P 21		2.70
8		P 3		2.68	8		P 18		2.69
9		P 2		2.68	9		P 17		2.67
10		P 13		2.07	10		P 16		2.59
11		P 14		2.07	11		P 12		2.12
12		P 11		2.07	12		P 9		2.11
13		P 15		2.06	13		P 10		2.10
14		P 10		2.06	14		P 15		2.10
15		P 9		2.05	15		P 14		2.09
16		P 12		2.05	16		P 13		2.09
17		P 8		2.00	17		P 11		2.09
18		P 17		1.47	18		P 8		2.05
19		P 18		1.47	19		P 2		1.48
20		P 16		1.47	20		P 3		1.48
21		P 21		1.46	21		P 1		1.47
22		P 22		1.46	22		P 4		1.44
23		P 20		1.45	23		P 5		1.44
24		P 23		1.45	24		P 0		1.41
25		P 19		1.44	25		P 7		1.41
26	pre_euler3d_cpu.exe (1836)			49.77	26		P 6		1.40
27	System (4)			0.11	27	System (4)			0.11
28	dwm.exe (5820)			0.11	28	dwm.exe (5820)			0.11
29	xperf.exe (6464)			0.03	29	xperf.exe (6464)			0.03

Figure 9. Continued ETL CPU Usage analysis

Overview of Intel VTune™ Profiler

Our VTune™ Profiler is a performance analysis tool used for many Intel applications. Using it helps in understanding how well or how poorly processes and functions are being handled in the performance hybrid architecture. VTune™ Profiler does this by finding serial and parallel code bottlenecks and speed execution. We recommend using this tool to analyze the algorithm choices and understand where and how your application can benefit from available hardware resources.

To get started, you can go to Intel.com and search keywords “VTune™ Installation”. After downloading, you will need to capture your Microarchitecture Exploration Analysis. This provides a complete microarchitectural analysis at the process/module/function level. Based on these results, we can capture other focused profiles for debugging. For example, you can run a 'Memory Access' analysis if the process is backend/memory-bound and has issues, or a 'Threading' analysis if concurrency is low. You can read more about this “top-down” microarchitecture by visiting www.intel.com and search “microarchitecture analysis”.

Below is a snapshot from VTune™:

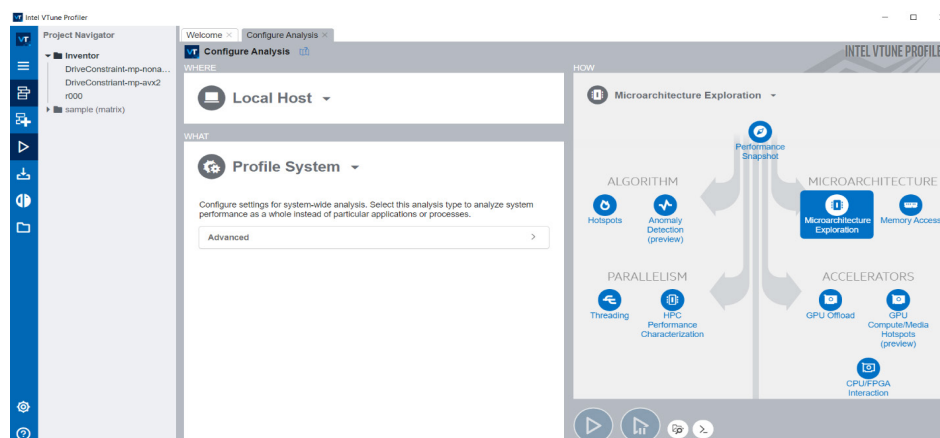


Figure 10. VTune™ Configure Window

Under the "What" pane one can select to profile at a System level or Process/application level, as shown below:

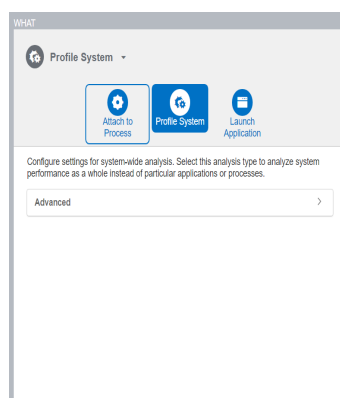


Figure 11. Vtune™ Profile System Window

Vtune™ Profiler for performance hybrid architecture

Our latest version of Vtune™ Profiler has support on the 12th Generation Intel Core processor platform. It allows group events and metrics in respect to core activity. As shown in the next figure, there is a pre-defined grouping available (which is also customizable).

Hardware Events									
Analysis Configuration Collection Log Summary Event Count Sample Count Caller/Callee Top-down Tree Platform									
Grouping: Core Type / Physical Core / Thread / Function / Call Stack									
Core Type / Physical Core / Thread / Function / Call Stack	Hardware Event Count by Hardware Event Type								
	INST	RETIRED.ANY	CPU	CLK	UNHALTED.THREAD	CPU	CLK	UNHALTED.REF	TSC
▼ Big Core		50,486,075,729			12,794,019,191			16,552,024,828	
▶ core_4		50,486,075,729			12,794,019,191			16,552,024,828	
▼ Small Core		2,678,004,017			0			2,774,004,161	
▶ core_0		982,001,473			0			904,001,356	
▶ core_1		734,001,101			0			674,001,011	
▶ core_2		658,000,987			0			848,001,272	
▶ core_3		304,000,456			0			348,000,522	

Figure 12. Core type grouping in Vtune™ Profiler (Intel Corp.)

The figure below shows grouping capabilities for timeline view. The drop-down 'Process / Core Type' allows us to see what core type a process worked on and when the migration happened.

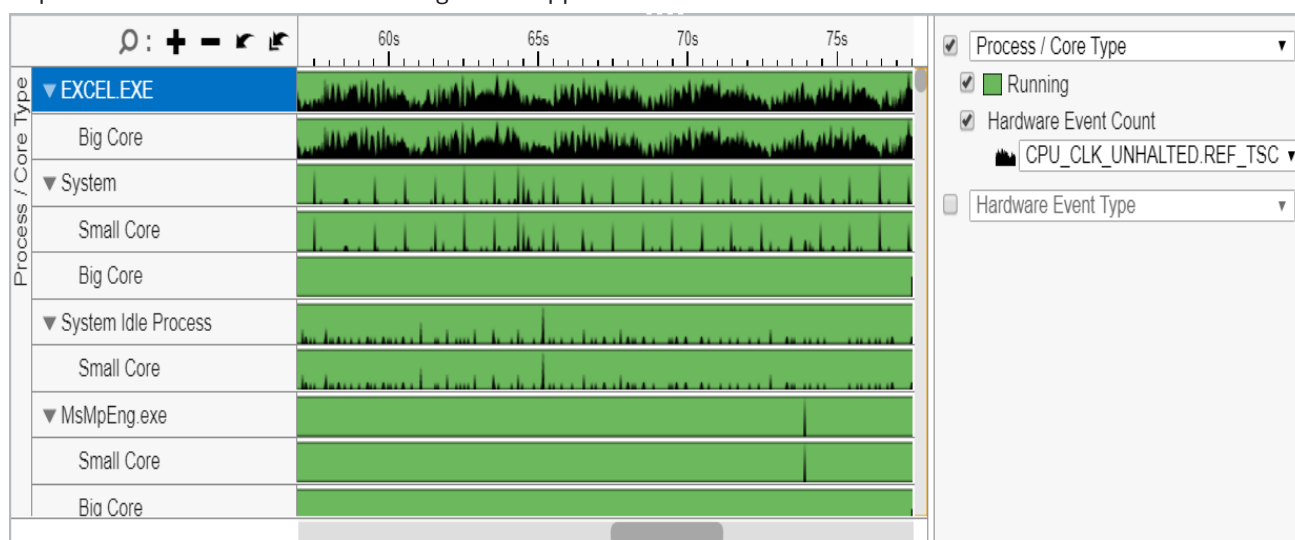


Figure 13. Continued Core type grouping in Vtune™ Profiler's timeline view (Intel Corp.)

Our Vtune™ Profiler also works through handling system hardware event-based sampling using the command-line tools SEP and EMON. These command-line tools collect data and sample events to showcase what is going on in the system.

Case Study 3: Top-down analysis with VTune™

For a famous creator workload, we noticed really good OOB performance gains on the 12th Generation Intel Core processor platform compared to the previous generation processor platform. To understand where these gains are coming from, we collected VTune™ 'Microarchitecture Exploration' analysis for the workload on both the current processor generation and the previous processor generation platforms to review the Top-down analysis.

As we see in the snapshot below, the generation-over-generation performance improvement in the 12th Generation Intel Core processor platform is achieved mainly due to:

- **Multi-core scaling:** Core bound % stat comparison in the snapshot below shows benefits on the 12th Generation Intel Core processor platform with an increase in the number of cores.
- **Improved micro-architecture:** IPC gains with improved microarchitecture translated into improved CPI on 12th Generation Intel Core processors.
- **Increase in cache size:** Bigger caches on the 12th Generation Intel Core processors improved cache locality and resulted in reduced data access latency as we can see in increased L3 Bound % and reduced DRAM Bound % for the 12th Generation Intel Core processors.

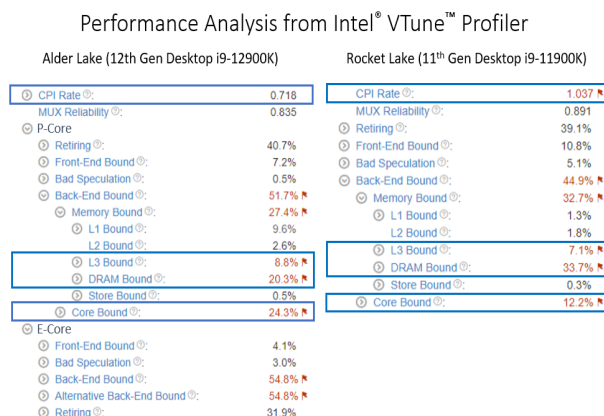


Figure 14. Top-down analysis in VTune™ Profiler's summary view (Intel Corp.)

Conclusion

In this paper, we discussed different issues and problems to keep in mind when debugging the performance issues that come with implementing performance hybrid architecture. There are several options to diagnose and prepare for a multitude of different issues and problems that can come from debugging, specifically focusing on WPA and VTune™ Profiler.

If you want to read more about Intel and what we are doing in the performance hybrid architecture space, please refer to our main site.

Disclaimer

Notice: This document contains information on products in the design phase of development. The information here is subject to change without notice. Do not finalize a design with this information.

References

- [Intel Architecture Instruction Set Extensions Programming Reference](#)
- [Windows Performance Recorder | Microsoft Docs](#)
- [Windows Performance Analyzer | Microsoft Docs](#)
- [typeperf | Microsoft Docs](#)
- [SetThreadInformation function \(processthreadsapi.h\) - Win32 apps | Microsoft Docs](#)
- [SetProcessInformation function \(processthreadsapi.h\) - Win32 apps | Microsoft Docs](#)
- [SetProcessAffinityMask function \(winbase.h\) - Win32 apps | Microsoft Docs](#)



Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex.

Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

All product plans and roadmaps are subject to change without notice.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Intel does not control or audit third-party benchmark data or the web sites referenced in this document. You should visit the referenced web site and confirm whether referenced data are accurate.

No product or component can be absolutely secure. Intel does not control or audit third-party data. You should consult other sources to evaluate accuracy.

Your costs and results may vary. Intel technologies may require enabled hardware, software or service activation.

© Intel Corporation.

Intel®, the Intel® logo, and other Intel marks are trademarks of Intel® Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.